

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	B.MANU JASWANTH	Reg. No.:	192525400
Section / Batch:	2025	Date:	28/07/2023

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation. • Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach. • Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I B.MANU JASWANTH/192525400 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: MANU

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 (a): Digital Library – Linear Search

a) Explain how Linear Search works in this scenario

A digital library stores book records in an **unsorted** order because new books are frequently added, updated, or

removed. Since the records are not sorted, **Linear Search (Sequential Search)** is the most suitable searching technique.

Working Process

1. Start from the first book record.
2. Compare the search key (Book ID, Title, or ISBN) with the current record.
3. If the record matches, stop the search and return the book details.
4. Otherwise, move to the next record.
5. Repeat until the book is found or all records have been checked.

Example

Book IDs:

[B105, B210, B315, B420, B525]

Search for **B420**

Step Record Compared Result

1	B105	Not Found
2	B210	Not Found
3	B315	Not Found
4	B420	Found

Number of Comparisons = 4

Q1 (b): Analyze Best, Average, and Worst Case Performance

Case	Situation	Comparisons	Time Complexity
Best Case	Target is the first record	1	O(1)
Average Case	Target is near the middle	$n/2$	O(n)
Worst Case	Target is last or absent	n	O(n)

Analysis

- **Best Case:** The required book is located immediately.
- **Average Case:** About half the records are checked before finding the book.
- **Worst Case:** Every record must be examined.

Thus, Linear Search has a **linear growth in execution time** as the number of records increases.

Q1 (c): Justify Whether Sorting Would Improve Efficiency

Sorting the records would improve searching efficiency because faster algorithms such as **Binary Search** could then be used.

Advantages of Sorting

- Binary Search reduces search time from **O(n)** to **O(log n)**.
- Suitable for libraries where searches are frequent and updates are rare.
- Faster retrieval for large databases.

Disadvantages

- Frequent insertion or deletion requires re-sorting.
- Maintaining sorted order increases update cost.
- Sorting itself requires additional computation.

Justification

Since the library is updated frequently, maintaining sorted records is expensive. Therefore, **Linear Search is more**

practical for frequently changing data, while sorting is preferable only when searches greatly outnumber updates.

CODE:

```
#include <stdio.h>

int main()
{
    int books[] = {105, 210, 315, 420, 525};
    int n = 5, key, i, found = 0;

    printf("Book Records: ");
    for(i = 0; i < n; i++)
        printf("%d ", books[i]);

    printf("\nEnter Book ID to Search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        printf("\nStep %d: Compare %d with %d", i + 1, books[i], key);

        if(books[i] == key)
        {
            printf(" --> Match Found");
            found = 1;
            break;
        }
        else
        {
            printf(" --> Not Matched");
        }
    }

    if(found)
        printf("\n\nBook Found at Position %d", i + 1);
    else
        printf("\n\nBook Not Found");

    printf("\nTotal Comparisons = %d\n", i + 1);

    return 0;
}
```

programiz.com/c-programming/online-compiler/

Programiz C Online Compiler

Programiz PRO

main.c

1 #include <stdio.h>

2

3 int main()

4 {

5 int books[] = {105, 210, 315, 420, 525};

6 int n = 5, key, i, found = 0;

7

8 printf("Book Records: ");

9 for(i = 0; i < n; i++)

10 printf("%d ", books[i]);

11

12 printf("\nEnter Book ID to Search: ");

13 scanf("%d", &key);

14

15 for(i = 0; i < n; i++)

16 {

17 printf("\nStep %d: Compare %d with %d", i + 1, books[i], key);

18

19 if(books[i] == key)

20 {

21 printf(" --> Match Found");

22 found = 1;

23 break;

24 }

25 } else

26 {

Output

Clear

Book Records: 105 210 315 420 525

Enter Book ID to Search: 420

Step 1: Compare 105 with 420 --> Not Matched

Step 2: Compare 210 with 420 --> Not Matched

Step 3: Compare 315 with 420 --> Not Matched

Step 4: Compare 420 with 420 --> Match Found

Book Found at Position 4

Total Comparisons = 4

=== Code Execution Successful ===

32°C Mostly cloudy

Search

ENG IN

10:10 AM 28-07-2026